

08 — Roadmap triển khai & Script trình bày với team

Trạng thái hiện tại (đối chiếu code thật trong repo)

Chi tiết đầy đủ (bug đã gặp + code + test thật): TuDocs/timesheet-guide/giai-doan-1-log/ (Log Work cơ bản), TuDocs/timesheet-guide/giai-doan-2-log/ (24h Cap + Race Condition), TuDocs/timesheet-guide/giai-doan-3-log/ (Time Lock API + Lớp phòng thủ 1).

Thành phần	Trạng thái
Models (TimeLock, LogWork, DailyUserTimesheet)	✅ Đã viết xong trong backend/timesheets/models.py
CheckConstraint(total_hours<=24), UniqueConstraint	✅ Đã có trong model
Migration đã chạy	✅ Đã xong
serializers_employee.py — EmployeeLogWorkSerializer	✅ Giai đoạn 1 xong (chứa Time Lock/24h-cap)
views_employee.py — EmployeeLogWorkView	✅ Giai đoạn 1 xong
urls_employee.py — POST /api/timesheets/log-works/	✅ Giai đoạn 1 xong
Permission timesheet:create (EMPLOYEE), timesheet:lock (MANAGER)	✅ Đã seed sẵn từ accounts/migrations/0002 — dùng lại được ngay
Data Isolation (Employee chỉ log work vào Task của chính mình)	✅ Giai đoạn 1 xong (validate_task)
Logic Pessimistic Locking (select_for_update) + Daily 24h Cap	✅ Giai đoạn 2 xong — test race condition thật (2 request song song) đã xác nhận
Time Lock API (ManagerTimeLockView) — chốt sổ kỳ báo cáo	✅ Giai đoạn 3 xong (chỉ chiều khóa, chưa Unlock)
Time Lock check (Lớp phòng thủ 1) trong Log Work	✅ Giai đoạn 4 xong

Câu hỏi nghiệp vụ cần chốt với team

(Unlock API? Override? Task

✗ Chưa hỏi (xem file 04, 07)

CANCELLED?)

Thứ tự code hợp lý (roadmap)

- ✓ Giai đoạn 1 – Log Work cơ bản (chưa có defensive logic) – >
 1. ✓ Serializer LogWork (validate format: hours_spent > 0,
 2. ✓ View tạo log_work đơn giản – CHƯA check Time Lock, CHƯA
 3. ✓ urls.py: POST /api/timesheets/log-works/
 4. ✓ Test bằng curl: tạo log_work thành công, dữ liệu lưu c
(4/4 test case – chi tiết `giai-doan-1-log/05-testing-va-

→ Mục đích giai đoạn này: có khung API chạy được trước, để c
thêm defensive logic vào mà không phải vừa nghĩ luồng vừa

- ✓ Giai đoạn 2 – Thêm Lớp phòng thủ 2 (Daily 24h Cap + Race Co
 5. ✓ Thêm transaction.atomic() + select_for_update() vào cr
 6. ✓ get_or_create() cho daily_user_timesheets nếu chưa có
 7. ✓ Logic cộng dồn + kiểm tra > 24h → ValidationError (400
 8. ✓ Test: 2 request curl chạy song song thật (không phải c
nhận race condition không xảy ra – (4/4 test case, chi ti
`giai-doan-2-log/02-testing-va-ket-qua.md`)

- ✓ Giai đoạn 3 – Time Lock (API chốt sổ) – XONG
 9. ✓ Serializer + View tạo TimeLock (chỉ chiều khóa, chưa l
 10. ✓ urls.py: POST /api/timesheets/time-locks/
 11. ✓ Permission check: chỉ role có "timesheet:lock" (dùng
class đã làm ở auth-guide)
 12. ✓ Test: khóa 1 kỳ, xác nhận log_work cho kỳ đó bị chặn

- ✓ Giai đoạn 4 – Thêm Lớp phòng thủ 1 vào Log Work (Time Lock
 13. ✓ Thêm bước kiểm tra TimeLock vào ĐẦU create() (Seriali
 14. ✓ Test: log work vào kỳ đã khóa → nhận đúng lỗi 403
(7/7 test case cho Giai đoạn 3-4 – chi tiết
`giai-doan-3-log/03-testing-va-ket-qua.md`)

Giai đoạn 5 – Timesheet Review (phía Manager)

15. API GET log-work có filter theo employee_id, date_from,
16. Áp dụng Data Isolation: chỉ trả về nhân viên thuộc team
request.user (Manager)

Giai đoạn 6 – Edge cases (làm sau khi 1-5 chạy ổn, xem chi tiế

17. API sửa/xóa log_work (nếu nghiệp vụ cần) – áp dụng lại r
+ time lock check tương tự lúc tạo
18. Xử lý exception RESTRICT khi xóa Task có log work, trả l
19. Chốt với team: Unlock API? Override cho Admin? Task CANC

Phụ thuộc chéo cần lưu ý

- Giai đoạn 3, 11 cần **Permission/RBAC đã code xong ở app accounts** (auth-guide file 04, 08 — Giai đoạn 3 ở đó). Vì cùng 1 người (bạn) làm cả 2 app, nên thực ra bạn có thể chủ động làm RBAC engine chung trước, rồi áp dụng lại ở cả accounts và timesheets — tránh viết permission check 2 lần theo 2 kiểu khác nhau.
- Giai đoạn 18 cần biết rõ **API xóa Task** (app tasks, người khác code) xử lý exception thế nào — nên trao đổi sớm, không để tới lúc test mới phát hiện 2 bên hiểu khác nhau.

Script gợi ý khi trình bày với team

"Phần timesheets của em giải quyết bài toán khác với phần accounts — ở đây không phải bảo mật ai-được-làm-gì, mà là đảm bảo **dữ liệu đúng khi nhiều request ghi đồng thời**, vì đây là dữ liệu dùng để tính lương.

Vấn đề cụ thể là Race Condition: nếu 2 request cùng đọc tổng giờ cũ, cùng cộng thêm, cùng ghi lại — request sau có thể ghi đè mất kết quả của request trước mà không ai biết, và tổng giờ cuối cùng sai. Em giải quyết bằng Pessimistic Locking — dùng `SELECT FOR UPDATE` để khóa cứng dòng dữ liệu tổng hợp của (nhân viên, ngày) đó trong lúc tính toán, request thứ 2 phải chờ request thứ 1 ghi xong rồi mới được đọc giá trị mới nhất.

Về mặt thiết kế bảng, em tách riêng bảng chi tiết `log_works` và bảng tổng hợp `daily_user_timesheets` — vì khóa 1 dòng tổng hợp duy nhất rẻ và an toàn hơn nhiều so với khóa một phép tính SUM trên nhiều dòng chi tiết.

Có 2 lớp chặn độc lập: kỳ báo cáo đã khóa (Time Lock) và tổng giờ vượt 24h — 2 loại lỗi này em trả 2 mã HTTP khác nhau (403 vs 400) vì bản chất khác nhau: một là bị cấm vì lý do hành chính, một là do chính dữ liệu nhập sai.

Em cũng đã rà một số edge case dễ bị bỏ sót như sửa/xóa log work cũ phải áp dụng lại đúng logic phòng thủ này, không chỉ API tạo mới — và còn vài câu hỏi nghiệp vụ cần team chốt trước khi code, ví dụ có cần API mở khóa lại kỳ báo cáo hay không."

Checklist tổng hợp

Bắt buộc (theo tài liệu yêu cầu)

- ✓ API Log Work (chấm công) — Giai đoạn 1
- ✓ Lớp phòng thủ 1: chặn nhập vào kỳ đã khóa — Giai đoạn 4

- ☒ Lớp phòng thủ 2: chặn vượt 24h/ngày, xử lý đúng Race Condition bằng Pessimistic Locking — Giai đoạn 2
- ☒ API chốt sổ kỳ báo cáo (Time Lock) — Giai đoạn 3 (chỉ chiều khóa)
- ☐ Double Submit prevention ở Frontend
- ☐ Filter theo Date Range + Employee cho trang Review (Manager) — có thể đã chuyển phạm vi sang Đức Long theo v2 (FR-124), cần xác nhận
- ☐ Data Isolation: Manager chỉ xem được team mình

Cần chốt với team trước khi code (nâng cao / chưa rõ trong tài liệu)

- ☐ API Unlock (mở khóa lại kỳ báo cáo)?
- ☐ Admin override Time Lock trong trường hợp đặc biệt?
- ☐ Task CANCELLED còn cho log work không?
- ☐ Time Lock khóa theo toàn công ty hay theo từng team/Manager?

Bước tiếp theo

Khi bạn đã đọc hết 8 file (cả series timesheet-guide và auth-guide) và trả lời được các câu hỏi tự kiểm tra, quay lại nhờ tôi triển khai code thật theo đúng thứ tự ở "Giai đoạn 1" — bắt đầu từ Serializer + View Log Work cơ bản.